In [27]:
```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
```

Task 1: Data Cleaning and Preparation

In [2]:
```
## Loading the data set structure
df = pd.read_csv("Target.csv")

# View first 5 rows
df.head()
```

Out[2]:

	Timestamp	age	Gender	Purchase_Frequency	Purchase_Categories	Personalized_Rec
0	2023/06/08 5:20:57 PM GMT+5:30	24	Others	Once a month	Clothing and Fashion;others	
1	2023/06/12 2:44:47 PM GMT+5:30	12	Male	Multiple times a week	Groceries and Gourmet Food;Beauty and Personal...	
2	2023/06/09 2:39:16 PM GMT+5:30	14	Prefer not to say	Few times a month	Clothing and Fashion;Home and Kitchen	
3	2023/06/08 7:33:15 AM GMT+5:30	51	Female	Multiple times a week	Beauty and Personal Care;Clothing and Fashion;...	
4	2023/06/11 11:01:04 PM GMT+5:30	46	Male	Multiple times a week	Groceries and Gourmet Food;Beauty and Personal...	

5 rows × 24 columns

In [3]:
```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 800 entries, 0 to 799
Data columns (total 24 columns):
 #   Column                                  Non-Null Count  Dtype
---  ------                                  --------------  -----
 0   Timestamp                               800 non-null    object
 1   age                                     800 non-null    int64
 2   Gender                                  800 non-null    object
 3   Purchase_Frequency                      800 non-null    object
 4   Purchase_Categories                     800 non-null    object
 5   Personalized_Recommendation_Frequency   800 non-null    object
 6   Browsing_Frequency                      800 non-null    object
 7   Product_Search_Method                   620 non-null    object
 8   Search_Result_Exploration               800 non-null    object
 9   Customer_Reviews_Importance             800 non-null    int64
 10  Add_to_Cart_Browsing                    800 non-null    object
 11  Cart_Completion_Frequency               800 non-null    object
 12  Cart_Abandonment_Factors                800 non-null    object
 13  Saveforlater_Frequency                  800 non-null    object
 14  Review_Left                             800 non-null    object
 15  Review_Reliability                      800 non-null    object
 16  Review_Helpfulness                      800 non-null    object
 17  Personalized_Recommendation_Frequency   800 non-null    int64
 18  Recommendation_Helpfulness              800 non-null    object
 19  Rating_Accuracy                         800 non-null    int64
 20  Shopping_Satisfaction                   800 non-null    int64
 21  Service_Appreciation                    800 non-null    object
 22  Improvement_Areas                       800 non-null    object
 23  transaction                             800 non-null    int64
dtypes: int64(6), object(18)
memory usage: 150.1+ KB
```

In [8]:
```
## Remove Duplicate Rows
df.duplicated().sum()
```

Out[8]:
```
np.int64(0)
```

In [ ]:
```
## no duplicates found.
```

In [ ]:
```
## Fixing Column Names
```

In [6]:
```
df.columns = df.columns.str.strip()
```

In [7]:
```
df.columns
```

Out[7]:
```
Index(['Timestamp', 'age', 'Gender', 'Purchase_Frequency',
       'Purchase_Categories', 'Personalized_Recommendation_Frequency',
       'Browsing_Frequency', 'Product_Search_Method',
       'Search_Result_Exploration', 'Customer_Reviews_Importance',
       'Add_to_Cart_Browsing', 'Cart_Completion_Frequency',
       'Cart_Abandonment_Factors', 'Saveforlater_Frequency', 'Review_Left',
       'Review_Reliability', 'Review_Helpfulness',
       'Personalized_Recommendation_Frequency', 'Recommendation_Helpfulness',
       'Rating_Accuracy', 'Shopping_Satisfaction', 'Service_Appreciation',
       'Improvement_Areas', 'transaction'],
      dtype='object')
```

In [12]:
```
## Removing duplicate columns
df = df.loc[:, ~df.columns.duplicated()]
```

In [13]:
```
df.columns
```

Out[13]:
```
Index(['Timestamp', 'age', 'Gender', 'Purchase_Frequency',
       'Purchase_Categories', 'Personalized_Recommendation_Frequency',
       'Browsing_Frequency', 'Product_Search_Method',
       'Search_Result_Exploration', 'Customer_Reviews_Importance',
       'Add_to_Cart_Browsing', 'Cart_Completion_Frequency',
       'Cart_Abandonment_Factors', 'Saveforlater_Frequency', 'Review_Left',
       'Review_Reliability', 'Review_Helpfulness',
       'Recommendation_Helpfulness', 'Rating_Accuracy',
       'Shopping_Satisfaction', 'Service_Appreciation', 'Improvement_Areas',
       'transaction'],
      dtype='object')
```

In [ ]:
```
## Handle Missing Values
```

In [10]:
```
df.isnull().sum()
```

Out[10]:
```
Timestamp                                  0
age                                        0
Gender                                     0
Purchase_Frequency                         0
Purchase_Categories                        0
Personalized_Recommendation_Frequency      0
Browsing_Frequency                         0
Product_Search_Method                    180
Search_Result_Exploration                  0
Customer_Reviews_Importance                0
Add_to_Cart_Browsing                       0
Cart_Completion_Frequency                  0
Cart_Abandonment_Factors                   0
Saveforlater_Frequency                     0
Review_Left                                0
Review_Reliability                         0
Review_Helpfulness                         0
Personalized_Recommendation_Frequency      0
Recommendation_Helpfulness                 0
Rating_Accuracy                            0
Shopping_Satisfaction                      0
Service_Appreciation                       0
Improvement_Areas                          0
transaction                                0
dtype: int64
```

In [16]:
```
## Product_Search_Method having 180 missing values

df['Product_Search_Method'] = df['Product_Search_Method'].fillna('Unknown')
df = df.copy()
```

In [17]:
```
df['Product_Search_Method'].isnull().sum()
```

Out[17]:
```
np.int64(0)
```

In [18]:
```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 800 entries, 0 to 799
Data columns (total 23 columns):
 #   Column                                  Non-Null Count  Dtype
---  ------                                  --------------  -----
 0   Timestamp                               800 non-null    object
 1   age                                     800 non-null    int64
 2   Gender                                  800 non-null    object
 3   Purchase_Frequency                      800 non-null    object
 4   Purchase_Categories                     800 non-null    object
 5   Personalized_Recommendation_Frequency   800 non-null    object
 6   Browsing_Frequency                      800 non-null    object
 7   Product_Search_Method                   800 non-null    object
 8   Search_Result_Exploration               800 non-null    object
 9   Customer_Reviews_Importance             800 non-null    int64
 10  Add_to_Cart_Browsing                    800 non-null    object
 11  Cart_Completion_Frequency               800 non-null    object
 12  Cart_Abandonment_Factors                800 non-null    object
 13  Saveforlater_Frequency                  800 non-null    object
 14  Review_Left                             800 non-null    object
 15  Review_Reliability                      800 non-null    object
 16  Review_Helpfulness                      800 non-null    object
 17  Recommendation_Helpfulness              800 non-null    object
 18  Rating_Accuracy                         800 non-null    int64
 19  Shopping_Satisfaction                   800 non-null    int64
 20  Service_Appreciation                    800 non-null    object
 21  Improvement_Areas                       800 non-null    object
 22  transaction                             800 non-null    int64
dtypes: int64(5), object(18)
memory usage: 143.9+ KB
```

In [ ]:
```
## Convert Numeric Columns
```

In [19]:
```
cols = ['Customer_Reviews_Importance', 'Shopping_Satisfaction', 'Rating_Accuracy

for col in cols:
    df[col] = pd.to_numeric(df[col], errors='coerce')
```

In [20]:
```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 800 entries, 0 to 799
Data columns (total 23 columns):
 #   Column                                  Non-Null Count  Dtype
---  ------                                  --------------  -----
 0   Timestamp                               800 non-null    object
 1   age                                     800 non-null    int64
 2   Gender                                  800 non-null    object
 3   Purchase_Frequency                      800 non-null    object
 4   Purchase_Categories                     800 non-null    object
 5   Personalized_Recommendation_Frequency   800 non-null    object
 6   Browsing_Frequency                      800 non-null    object
 7   Product_Search_Method                   800 non-null    object
 8   Search_Result_Exploration               800 non-null    object
 9   Customer_Reviews_Importance             800 non-null    int64
 10  Add_to_Cart_Browsing                    800 non-null    object
 11  Cart_Completion_Frequency               800 non-null    object
 12  Cart_Abandonment_Factors                800 non-null    object
 13  Saveforlater_Frequency                  800 non-null    object
 14  Review_Left                             800 non-null    object
 15  Review_Reliability                      800 non-null    object
 16  Review_Helpfulness                      800 non-null    object
 17  Recommendation_Helpfulness              800 non-null    object
 18  Rating_Accuracy                         800 non-null    int64
 19  Shopping_Satisfaction                   800 non-null    int64
 20  Service_Appreciation                    800 non-null    object
 21  Improvement_Areas                       800 non-null    object
 22  transaction                             800 non-null    int64
dtypes: int64(5), object(18)
memory usage: 143.9+ KB
```

In [ ]:
```
## Convert Timestamp to datetime
```

In [22]:
```
df['Timestamp'] = pd.to_datetime(df['Timestamp'])
```

In [23]:
```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 800 entries, 0 to 799
Data columns (total 23 columns):
 #   Column                                Non-Null Count  Dtype
---  ------                                --------------  -----
 0   Timestamp                             800 non-null    datetime64[ns, UTC-0
5:30]
 1   age                                   800 non-null    int64
 2   Gender                                800 non-null    object
 3   Purchase_Frequency                    800 non-null    object
 4   Purchase_Categories                   800 non-null    object
 5   Personalized_Recommendation_Frequency 800 non-null    object
 6   Browsing_Frequency                    800 non-null    object
 7   Product_Search_Method                 800 non-null    object
 8   Search_Result_Exploration             800 non-null    object
 9   Customer_Reviews_Importance           800 non-null    int64
 10  Add_to_Cart_Browsing                  800 non-null    object
 11  Cart_Completion_Frequency             800 non-null    object
 12  Cart_Abandonment_Factors              800 non-null    object
 13  Saveforlater_Frequency                800 non-null    object
 14  Review_Left                           800 non-null    object
 15  Review_Reliability                    800 non-null    object
 16  Review_Helpfulness                    800 non-null    object
 17  Recommendation_Helpfulness            800 non-null    object
 18  Rating_Accuracy                       800 non-null    int64
 19  Shopping_Satisfaction                 800 non-null    int64
 20  Service_Appreciation                  800 non-null    object
 21  Improvement_Areas                     800 non-null    object
 22  transaction                           800 non-null    int64
dtypes: datetime64[ns, UTC-05:30](1), int64(5), object(17)
memory usage: 143.9+ KB
```

In [24]:
```
df.describe()
```

Out[24]:

	age	Customer_Reviews_Importance	Rating_Accuracy	Shopping_Satisfactio
count	800.000000	800.000000	800.000000	800.00000
mean	34.737500	3.003750	3.058750	2.93500
std	18.938306	1.431362	1.439753	1.43643
min	3.000000	1.000000	1.000000	1.00000
25%	18.000000	2.000000	2.000000	2.00000
50%	34.000000	3.000000	3.000000	3.00000
75%	50.250000	4.000000	4.000000	4.00000
max	67.000000	5.000000	5.000000	5.00000

In [25]:
```
df.head()
```

Out[25]:

	Timestamp	age	Gender	Purchase_Frequency	Purchase_Categories	Personalized_Rec
0	2023-06-08 17:20:57-05:30	24	Others	Once a month	Clothing and Fashion;others	
1	2023-06-12 14:44:47-05:30	12	Male	Multiple times a week	Groceries and Gourmet Food;Beauty and Personal...	
2	2023-06-09 14:39:16-05:30	14	Prefer not to say	Few times a month	Clothing and Fashion;Home and Kitchen	
3	2023-06-08 07:33:15-05:30	51	Female	Multiple times a week	Beauty and Personal Care;Clothing and Fashion;...	
4	2023-06-11 23:01:04-05:30	46	Male	Multiple times a week	Groceries and Gourmet Food;Beauty and Personal...	

5 rows × 23 columns

In [26]:
```
df.isnull().sum()
```

Out[26]:
```
Timestamp                                 0
age                                       0
Gender                                    0
Purchase_Frequency                        0
Purchase_Categories                       0
Personalized_Recommendation_Frequency     0
Browsing_Frequency                        0
Product_Search_Method                     0
Search_Result_Exploration                 0
Customer_Reviews_Importance               0
Add_to_Cart_Browsing                      0
Cart_Completion_Frequency                 0
Cart_Abandonment_Factors                  0
Saveforlater_Frequency                    0
Review_Left                               0
Review_Reliability                        0
Review_Helpfulness                        0
Recommendation_Helpfulness                0
Rating_Accuracy                           0
Shopping_Satisfaction                     0
Service_Appreciation                      0
Improvement_Areas                         0
transaction                               0
dtype: int64
```

Task 2: Descriptive Behavior Analysis

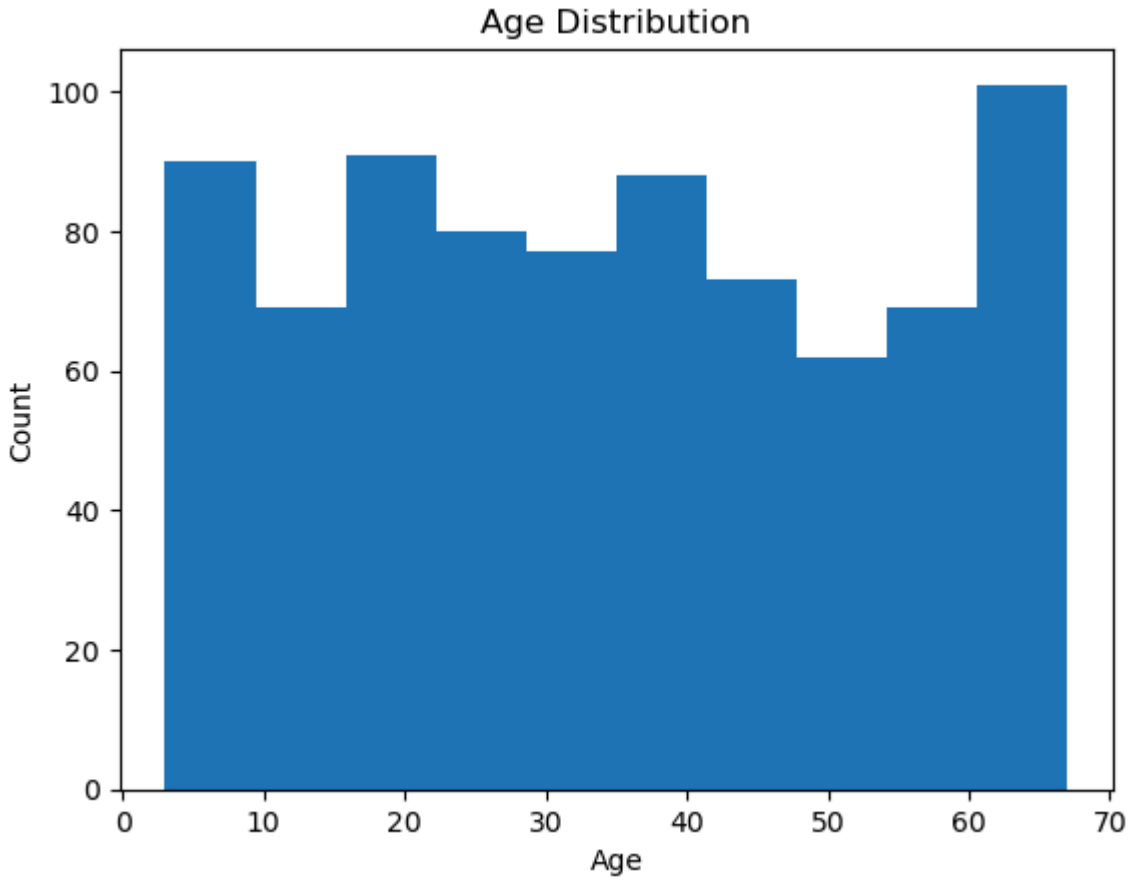
a.Summarize customer demographics

In [28]:
```
## Age Distribution
df['age'].describe()
```

Out[28]:
```
count    800.000000
mean      34.737500
std       18.938306
min        3.000000
25%       18.000000
50%       34.000000
75%       50.250000
max       67.000000
Name: age, dtype: float64
```

In [ ]:
```
## Visualization
## To understand which age group shops more
```

In [29]:
```
plt.hist(df['age'])
plt.title("Age Distribution")
plt.xlabel("Age")
plt.ylabel("Count")
plt.show()
```



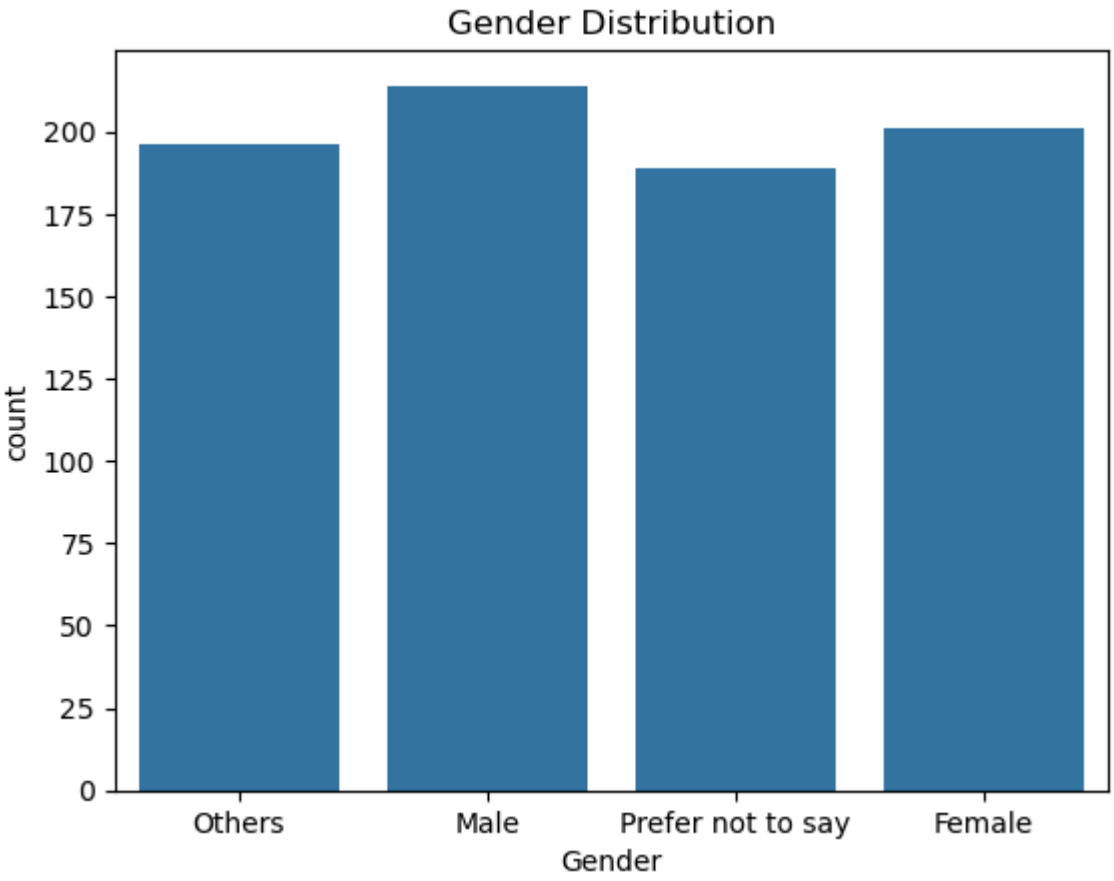
Insights a. The age distribution is fairly spread across all age groups, indicating a diverse customer base. b.There is no extreme concentration in a single age group, suggesting that Target attracts customers from young to older age segments.

In [ ]:
```
## Gender Distribution
```

In [30]:
```
df['Gender'].value_counts()
```

Out[30]:
```
Gender
Male                 214
Female               201
Others               196
Prefer not to say    189
Name: count, dtype: int64
```

In [31]:
```
sns.countplot(x='Gender', data=df)
plt.title("Gender Distribution")
plt.show()
```



Insights a.The dataset shows a balanced distribution across genders, with slightly higher participation from males. b.A noticeable portion of users also selected “Prefer not to say” and “Others,” indicating inclusivity in customer demographics.

b.Analyze overall purchase frequency and most popular product categories.

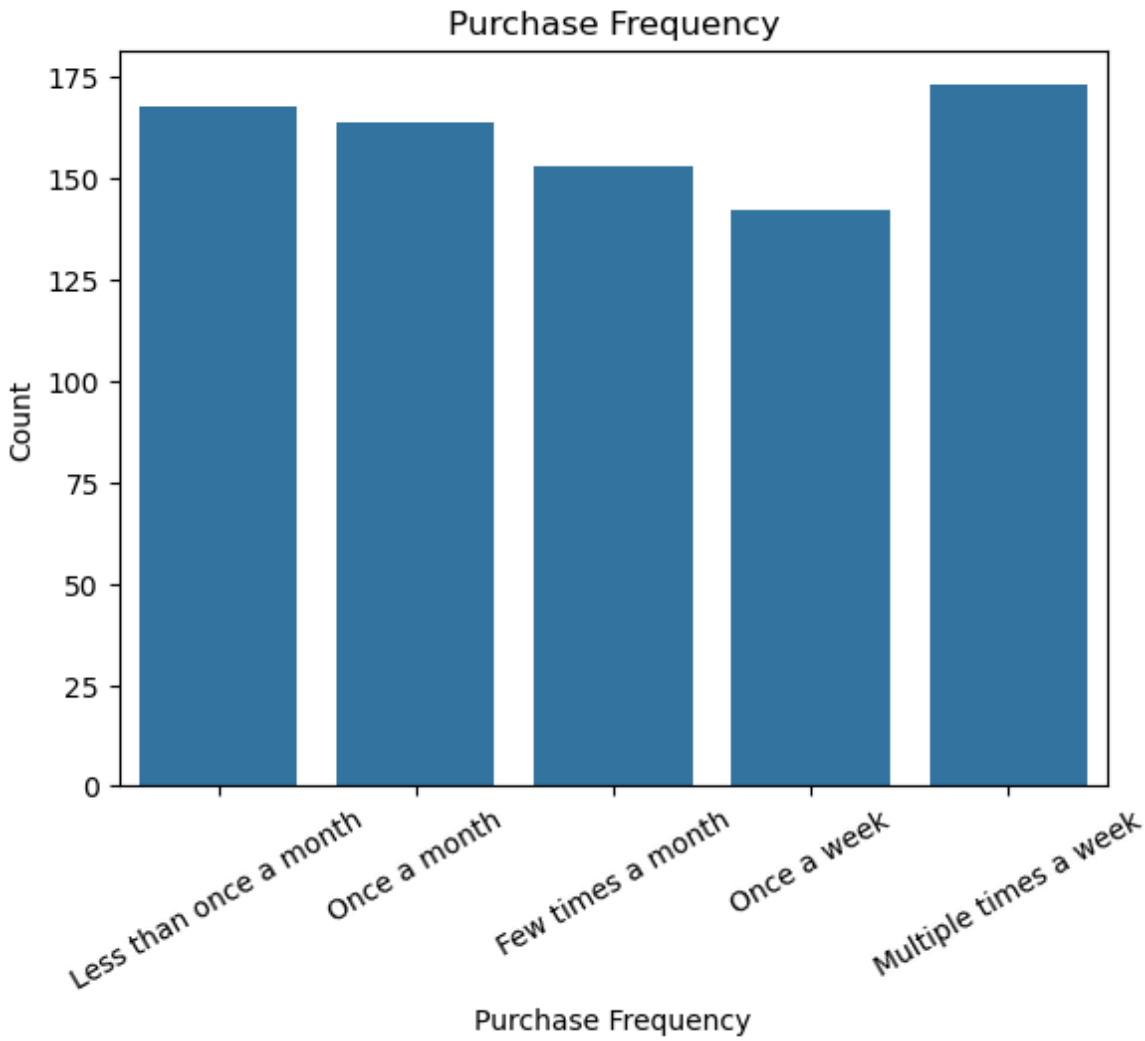
In [32]:
```
df['Purchase_Frequency'].value_counts()
```

Out[32]:
```
Purchase_Frequency
Multiple times a week     173
Less than once a month    168
Once a month              164
Few times a month         153
Once a week               142
Name: count, dtype: int64
```

In [37]:
```
order = [
    'Less than once a month',
    'Once a month',
    'Few times a month',
```

```
    'Once a week',
    'Multiple times a week'
]
```

In [38]:
```
sns.countplot(x='Purchase_Frequency', data=df, order=order)

plt.title("Purchase Frequency")
plt.xticks(rotation=30)   # rotate labels
plt.xlabel("Purchase Frequency")
plt.ylabel("Count")

plt.show()
```



In [39]:
```
df['Purchase_Frequency'].value_counts(normalize=True) * 100
```

Out[39]:
```
Purchase_Frequency
Multiple times a week     21.625
Less than once a month    21.000
Once a month              20.500
Few times a month         19.125
Once a week               17.750
Name: proportion, dtype: float64
```

Insights a.A significant number of customers shop frequently, with “Multiple times a week” being the most common behavior. b.However, a large portion also shops infrequently, indicating a mix of both loyal and occasional customers.

Most Popular Product Categories

In [40]:
```
df['Purchase_Categories'] = df['Purchase_Categories'].str.split(';')
```

In [41]:
```
df_exploded = df.explode('Purchase_Categories')
```

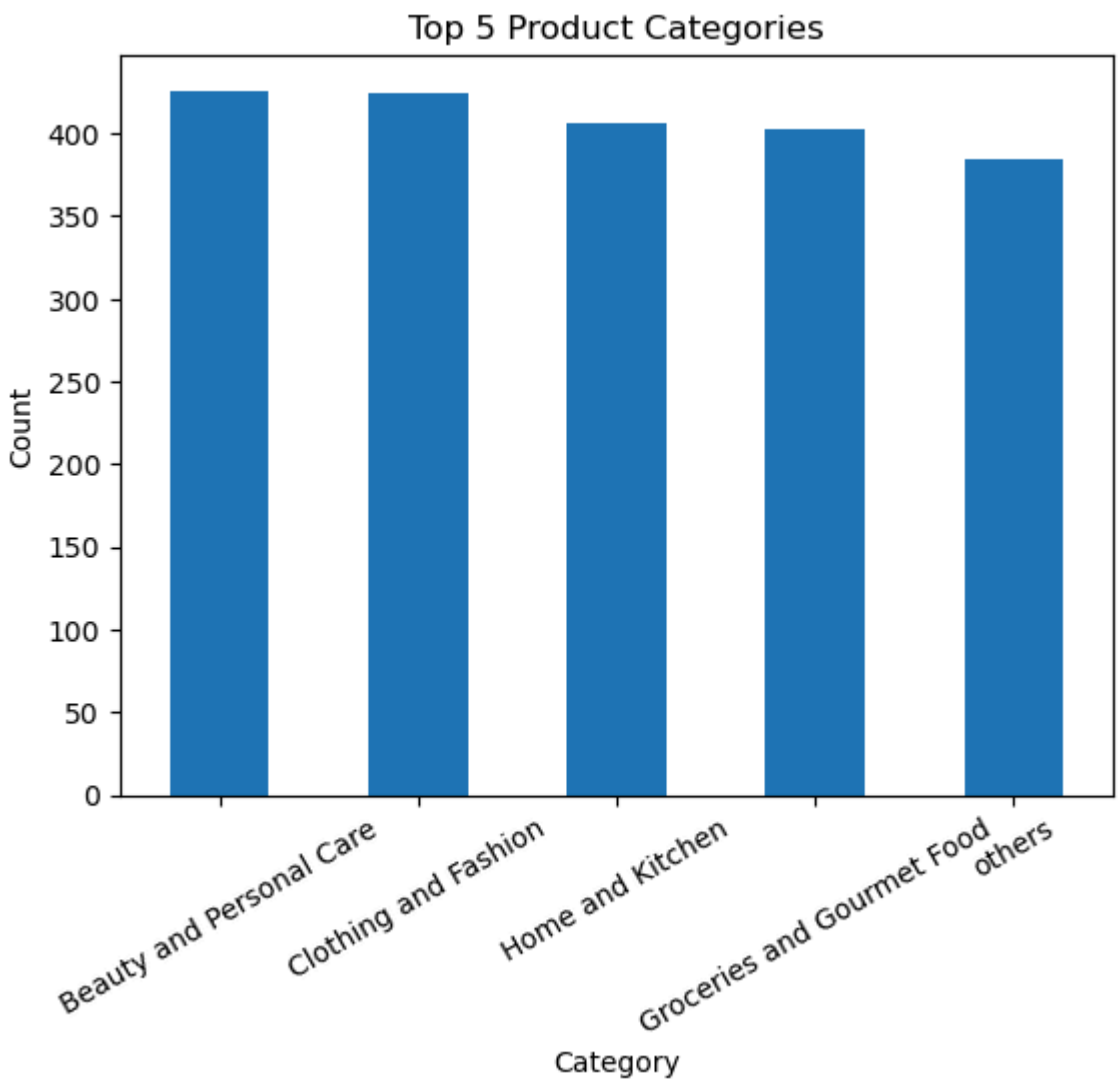
In [42]:
```
df_exploded['Purchase_Categories'].value_counts()
```

Out[42]:
```
Purchase_Categories
Beauty and Personal Care      426
Clothing and Fashion          424
Home and Kitchen              406
Groceries and Gourmet Food    403
others                        384
Name: count, dtype: int64
```

In [45]:
```
df['Purchase_Categories'].value_counts().head(10)
```

Out[45]:
```
Purchase_Categories
[Groceries and Gourmet Food, Beauty and Personal Care, others]
37
[Groceries and Gourmet Food, Beauty and Personal Care, Clothing and Fashion, Ho
me and Kitchen, others]    34
[Home and Kitchen]
33
[Groceries and Gourmet Food, Beauty and Personal Care]
32
[Beauty and Personal Care, Clothing and Fashion]
32
[Groceries and Gourmet Food, Clothing and Fashion, others]
32
[Groceries and Gourmet Food]
31
[Beauty and Personal Care, Home and Kitchen]
31
[others]
30
[Clothing and Fashion, others]
29
Name: count, dtype: int64
```

In [46]:
```
df_exploded['Purchase_Categories'].value_counts().head(5).plot(kind='bar')

plt.title("Top 5 Product Categories")
plt.xlabel("Category")
plt.ylabel("Count")
plt.xticks(rotation=30)

plt.show()
```



Insights a.Beauty and Personal Care and Clothing and Fashion are the most popular product categories among customers. b.Groceries and Gourmet Food also show high demand, indicating frequent purchase of essential items c.Home and Kitchen has slightly lower engagement, suggesting it is a secondary purchase category. d.Customers often purchase across multiple categories, highlighting strong cross-selling opportunities.

c.Identify top browsing methods and most common cart abandonment factors.

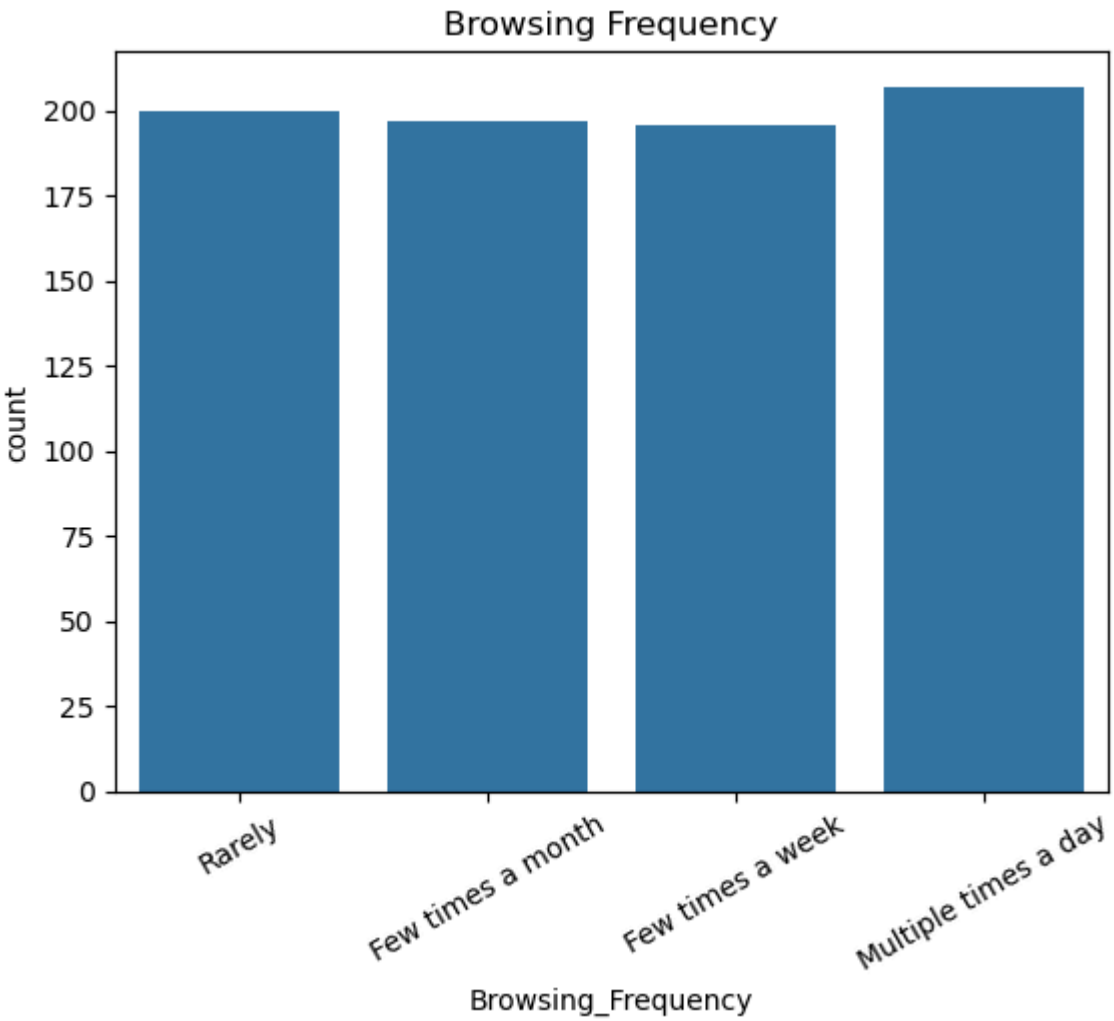
In [ ]:
```
## Browsing Frequency
```

In [47]:
```
df['Browsing_Frequency'].value_counts()
```

Out[47]:
```
Browsing_Frequency
Multiple times a day    207
Rarely                  200
Few times a month       197
Few times a week        196
Name: count, dtype: int64
```

In [61]:
```
order = ['Rarely', 'Few times a month', 'Few times a week', 'Multiple times a da

sns.countplot(x='Browsing_Frequency', data=df, order=order)
plt.title("Browsing Frequency")
```

```
plt.xticks(rotation=30)
plt.show()
```



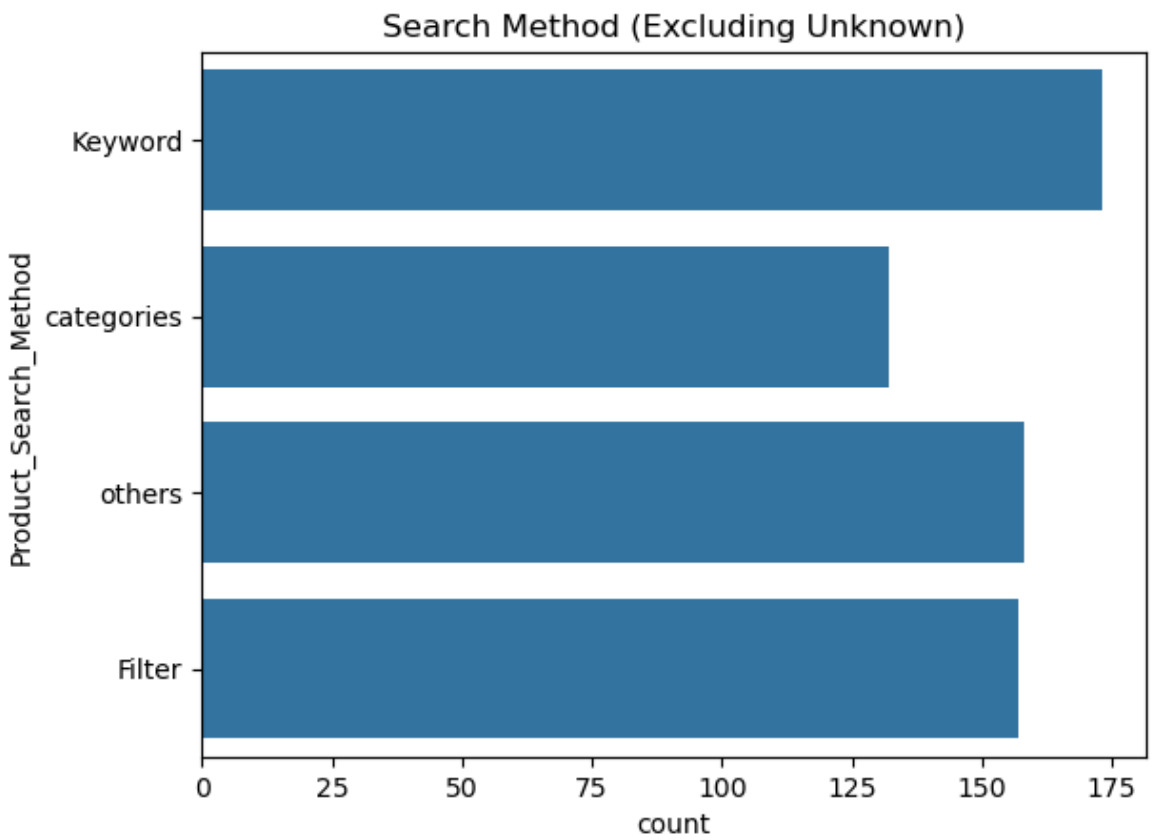
Insights a. Customers show active engagement, with “Multiple times a day” being slightly higher than other categories. b. Browsing behavior is fairly evenly distributed, indicating that users interact with the platform regularly at different frequencies.

In [ ]:
```
## Product Search Method
```

In [52]:
```
df['Product_Search_Method'].value_counts()
```

Out[52]:
```
Product_Search_Method
Unknown       180
Keyword       173
others        158
Filter        157
categories    132
Name: count, dtype: int64
```

In [62]:
```
df_filtered = df[df['Product_Search_Method'] != 'Unknown']

sns.countplot(y='Product_Search_Method', data=df_filtered)

plt.title("Search Method (Excluding Unknown)")
plt.show()
```



Insights a.Keyword search is one of the most preferred methods, indicating that users actively search for specific products. c.Filters and category browsing are also commonly used, highlighting the importance of navigation features.

Cart Behaviour

In [ ]:
```
### Cart Completion Frequency
```

In [54]:
```
df['Cart_Completion_Frequency'].value_counts()
```

Out[54]:
```
Cart_Completion_Frequency
Rarely       180
Sometimes    160
Always       159
Often        158
Never        143
Name: count, dtype: int64
```

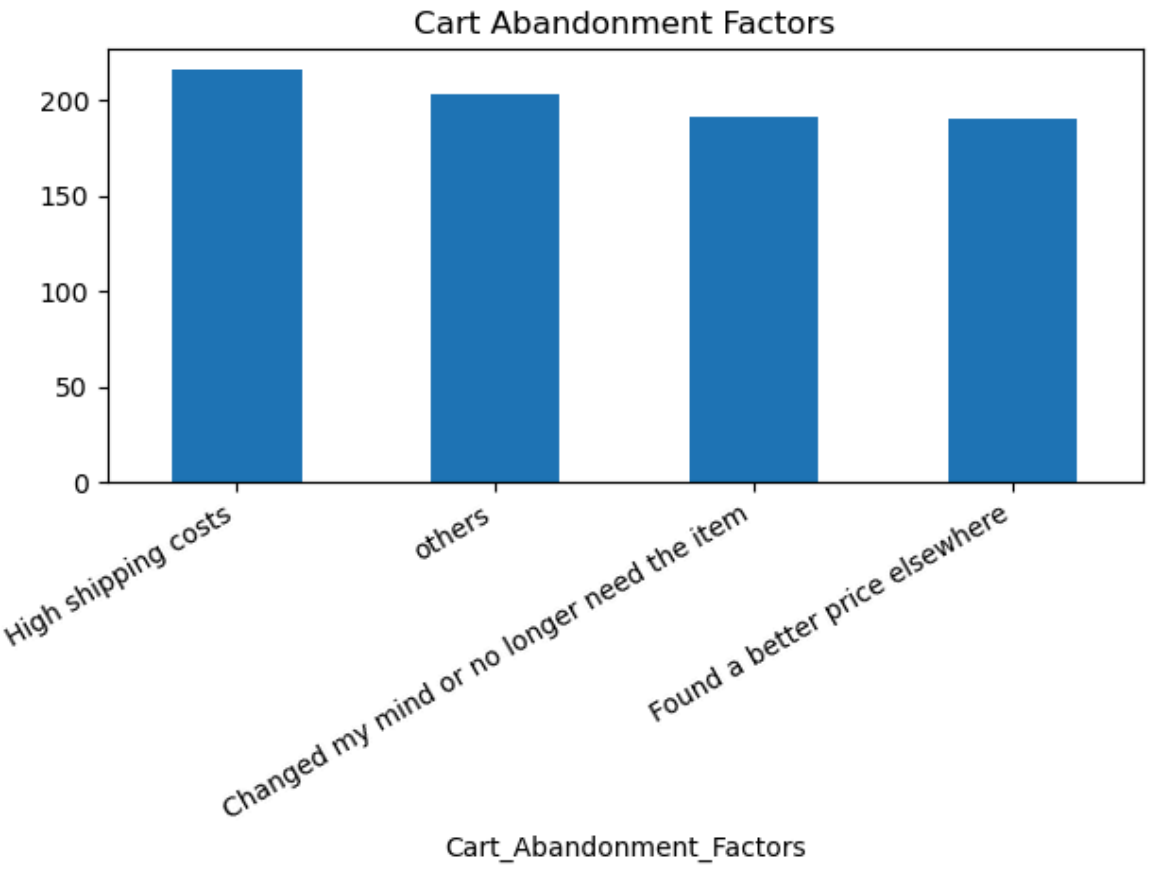
In [ ]:
```
### Cart Abandonment Reasons
```

In [55]:
```
df['Cart_Abandonment_Factors'].value_counts().head(5)
```

Out[55]:
```
Cart_Abandonment_Factors
High shipping costs                          216
others                                       203
Changed my mind or no longer need the item   191
Found a better price elsewhere               190
Name: count, dtype: int64
```

In [69]:
```
ax = df['Cart_Abandonment_Factors'].value_counts().head(5).plot(kind='bar')

plt.title("Cart Abandonment Factors")
```

```
plt.xticks(rotation=30, ha='right')   # rotate + align right

plt.tight_layout()   # prevents cutting of labels
plt.show()
```



Insights a.High shipping costs are the leading reason for cart abandonment. b.Other factors like “finding better prices elsewhere” and “no longer needing the item” also contribute significantly. c.This indicates pricing and cost transparency are key areas for improvement.

d.Calculate mean and median satisfaction, recommendation helpfulness, and rating accuracy.

In [ ]:
```
## Calculating Mean and Median
```

In [57]:
```
df[['Shopping_Satisfaction', 'Rating_Accuracy', 'Recommendation_Helpfulness']].d
```

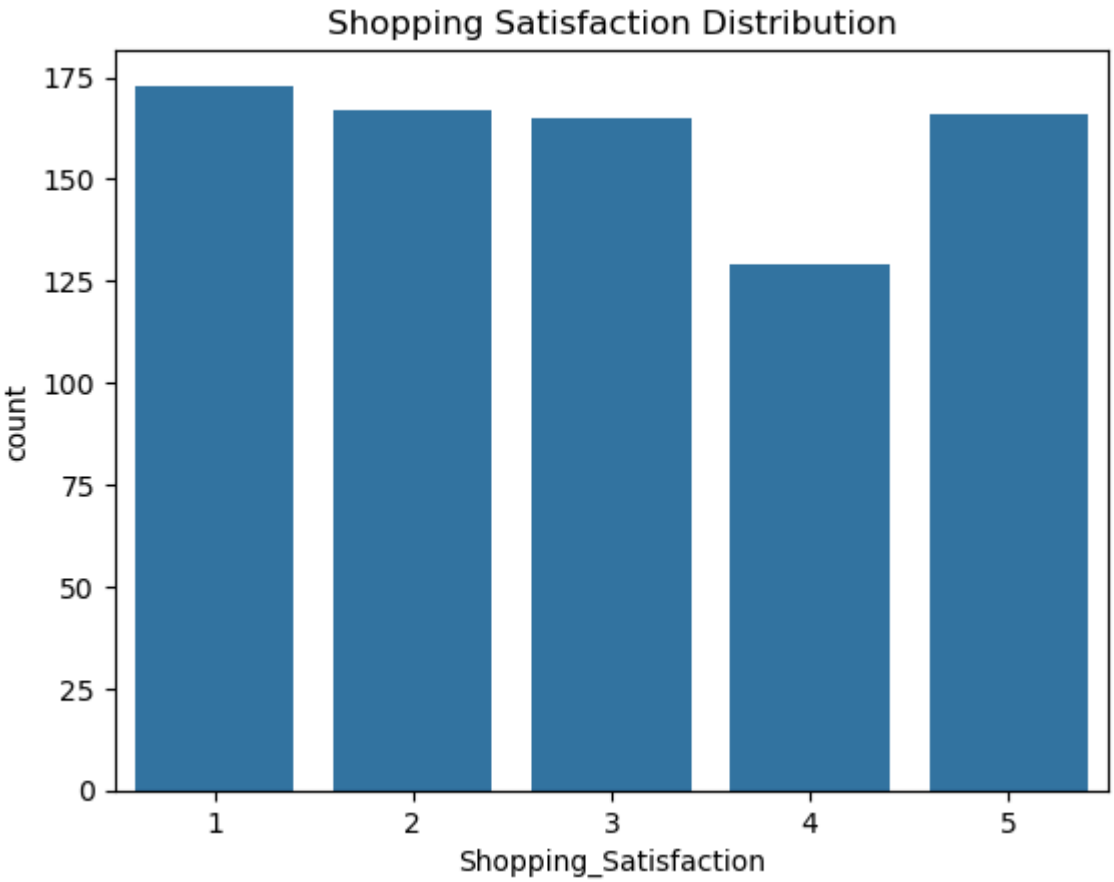
Out[57]:

	Shopping_Satisfaction	Rating_Accuracy
count	800.000000	800.000000
mean	2.935000	3.058750
std	1.436438	1.439753
min	1.000000	1.000000
25%	2.000000	2.000000
50%	3.000000	3.000000
75%	4.000000	4.000000
max	5.000000	5.000000

In [58]:
```
df['Shopping_Satisfaction'].mean()
df['Shopping_Satisfaction'].median()
```

Out[58]: 3.0

In [67]:
```
sns.countplot(x='Shopping_Satisfaction', data=df)

plt.title("Shopping Satisfaction Distribution")
plt.show()
```



Insights a.The average shopping satisfaction score is around 3, indicating moderate satisfaction among customers. b.The distribution shows responses spread across all levels, meaning both satisfied and dissatisfied customers exist. c.There is scope for improving customer experience to shift satisfaction towards higher ratings.

e.Generate summary statistics and visualizations for key behavioral variables.

In [60]:
```
df.describe()
```

Out[60]:

	age	Customer_Reviews_Importance	Rating_Accuracy	Shopping_Satisfactio
count	800.000000	800.000000	800.000000	800.00000
mean	34.737500	3.003750	3.058750	2.93500
std	18.938306	1.431362	1.439753	1.43643
min	3.000000	1.000000	1.000000	1.00000
25%	18.000000	2.000000	2.000000	2.00000
50%	34.000000	3.000000	3.000000	3.00000
75%	50.250000	4.000000	4.000000	4.00000
max	67.000000	5.000000	5.000000	5.00000

Insights a.The customer base is primarily young adults with moderate engagement levels. b.Customers show average trust in reviews and ratings, indicating potential to improve credibility. c.Overall satisfaction is neutral, suggesting room for enhancement in user experience.

Task 3: Customer Segmentation and Profiling

a.Segment customers based on purchase frequency and shopping satisfaction levels.

In [ ]:
```
## checking unique values
```

In [70]:
```
df['Purchase_Frequency'].unique()  ## shows all distinct values in a column
df['Shopping_Satisfaction'].unique()
```

Out[70]:
```
array([1, 3, 2, 4, 5])
```

In [ ]:
```
## Convert Purchase_Frequency to numeric
```

In [85]:
```
freq_map = {
    'Less than once a month': 1,
    'Once a month': 2,
    'Few times a month': 3,
    'Once a week': 4,
    'Multiple times a week': 5
}

df['Purchase_Frequency_Num'] = df['Purchase_Frequency'].map(freq_map) ## replace
```

In [86]:
```
df['Purchase_Frequency_Num'].value_counts()
```

Out[86]:
```
Purchase_Frequency_Num
5    173
1    168
2    164
3    153
4    142
Name: count, dtype: int64
```

In [ ]:
```
## Ensuring Shopping_Satisfaction is numeric
```

In [73]:
```
df['Shopping_Satisfaction'] = pd.to_numeric(df['Shopping_Satisfaction'], errors=

## pd.to_numeric() : converts column into numbers
## errors='coerce' : invalid values become NaN
```

In [ ]:
```
## Create Segments using conditions
```

In [88]:
```
def segment_customer(row):
    if row['Purchase_Frequency_Num'] >= 3 and row['Shopping_Satisfaction'] >= 3:
        return 'Frequent Buyers'
    elif row['Purchase_Frequency_Num'] >= 2 and row['Shopping_Satisfaction'] >=
        return 'Occasional Shoppers'
    else:
        return 'At-Risk Customers'
```

In [ ]:
```
## Analyzing segment distribution
```

In [89]:
```
df['Customer_Segment'] = df.apply(segment_customer, axis=1) ## applies function

df['Customer_Segment'].value_counts()  ## counts number of customers in each seg
```

Out[89]:
```
Customer_Segment
At-Risk Customers      305
Frequent Buyers        266
Occasional Shoppers    229
Name: count, dtype: int64
```

In [91]:
```
import seaborn as sns
import matplotlib.pyplot as plt

sns.countplot(x='Customer_Segment', data=df)
plt.title('Customer Segments Distribution')
plt.show()
```

In [93]:
```
df['Purchase_Frequency'].value_counts()
```

Out[93]:
```
Purchase_Frequency
Multiple times a week     173
Less than once a month    168
Once a month              164
Few times a month         153
Once a week               142
Name: count, dtype: int64
```

Insights The majority of customers fall into the At-Risk segment, indicating a significant portion of users have low satisfaction or engagement. Frequent Buyers form a strong customer base with high purchase frequency, while Occasional Shoppers represent potential customers who can be targeted with personalized promotions to increase engagement.

b.Create profiles such as: ○ Frequent Buyers: High purchase frequency, high satisfaction. ○ Occasional Shoppers: Medium frequency, moderate satisfaction. ○ At-Risk Customers: Low satisfaction or frequent cart abandonment.

In [ ]:
```
## Average satisfaction per segment
```

In [94]:
```
df.groupby('Customer_Segment')['Shopping_Satisfaction'].mean()
```

Out[94]:
```
Customer_Segment
At-Risk Customers      2.081967
Frequent Buyers        3.981203
Occasional Shoppers    2.855895
Name: Shopping_Satisfaction, dtype: float64
```

In [ ]:
```
## Cart Abandonment behavior
## Shows which segment abandons carts more
```

In [95]:
```
df.groupby('Customer_Segment')['Cart_Abandonment_Factors'].value_counts()
```

Out[95]:
```
Customer_Segment     Cart_Abandonment_Factors
At-Risk Customers    High shipping costs                          83
                     others                                       83
                     Found a better price elsewhere               70
                     Changed my mind or no longer need the item   69
Frequent Buyers      Changed my mind or no longer need the item   71
                     others                                       67
                     High shipping costs                          66
                     Found a better price elsewhere               62
Occasional Shoppers  High shipping costs                          67
                     Found a better price elsewhere               58
                     others                                       53
                     Changed my mind or no longer need the item   51
Name: count, dtype: int64
```

In [ ]:
```
## Review Behaviour
```

In [96]:
```
df.groupby('Customer_Segment')['Review_Left'].value_counts(normalize=True)
## normalize=True → gives percentage instead of count
```

Out[96]:
```
Customer_Segment     Review_Left
At-Risk Customers    Yes            0.518033
                     No             0.481967
Frequent Buyers      Yes            0.503759
                     No             0.496241
Occasional Shoppers  No             0.506550
                     Yes            0.493450
Name: proportion, dtype: float64
```

c.Analyze demographic or behavioral differences across these segments

In [ ]:
```
## Gender distribution
```

In [97]:
```
df.groupby('Customer_Segment')['Gender'].value_counts()
```

Out[97]:
```
Customer_Segment     Gender
At-Risk Customers    Prefer not to say    89
                     Female               81
                     Male                 72
                     Others               63
Frequent Buyers      Others               84
                     Male                 76
                     Female               61
                     Prefer not to say    45
Occasional Shoppers  Male                 66
                     Female               59
                     Prefer not to say    55
                     Others               49
Name: count, dtype: int64
```

In [ ]:
```
## Age comparison
```

In [98]:
```
df.groupby('Customer_Segment')['age'].mean()
```

Out[98]:
```
Customer_Segment
At-Risk Customers       34.209836
Frequent Buyers         36.045113
Occasional Shoppers     33.921397
Name: age, dtype: float64
```

d.Use clustering (e.g., K-Means) for behavioral grouping based on survey responses.

In [ ]:
```
## Select features
```

In [99]:
```
features = df[['Purchase_Frequency_Num', 'Shopping_Satisfaction']]
```

In [ ]:
```
## Handle missing values
```

In [100…
```
features = features.dropna()
```

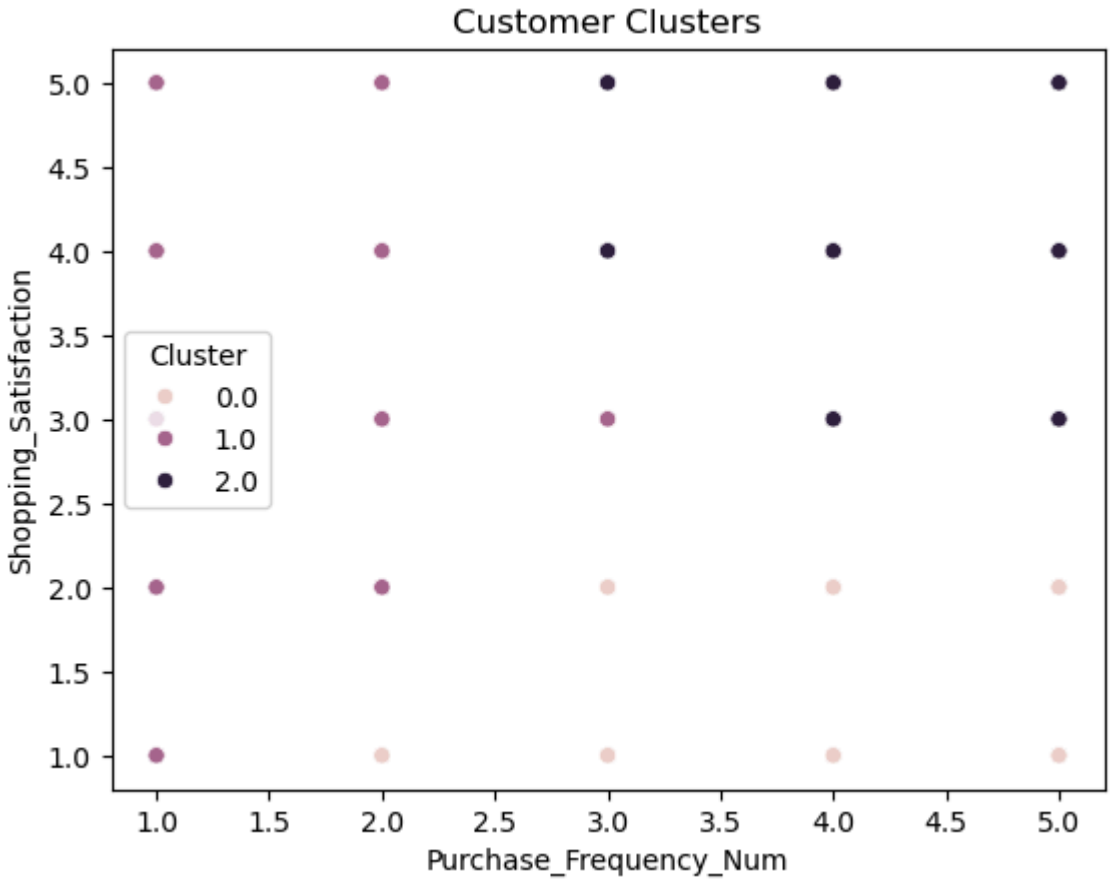
In [ ]:
```
## Apply K-Means
```

In [105…
```
from sklearn.cluster import KMeans

kmeans = KMeans(n_clusters=3, random_state=42)
df.loc[features.index, 'Cluster'] = kmeans.fit_predict(features)
```

```
C:\Users\afifa\anaconda3\Lib\site-packages\sklearn\cluster\_kmeans.py:1419: UserW
arning: KMeans is known to have a memory leak on Windows with MKL, when there are
less chunks than available threads. You can avoid it by setting the environment v
ariable OMP_NUM_THREADS=4.
  warnings.warn(
```

In [ ]:
```
## Visualize clusters
```

In [106…
```
sns.scatterplot(
    x='Purchase_Frequency_Num',
    y='Shopping_Satisfaction',
    hue='Cluster',
    data=df
)
plt.title('Customer Clusters')
plt.show()
```



In [ ]:
```
## Interpret Clusters
```

In [107…
```
df.groupby('Cluster')[['Purchase_Frequency_Num', 'Shopping_Satisfaction']].mean(
```

Out[107…

Cluster	Purchase_Frequency_Num	Shopping_Satisfaction
0.0	3.795652	1.404348
1.0	1.578313	3.162651
2.0	4.163866	4.096639

Customer Profiling Insight Frequent Buyers tend to be slightly older and more engaged, while At-Risk customers show lower satisfaction levels and mixed demographic distribution, indicating potential dissatisfaction across user groups.

Clustering Insight K-Means clustering reveals three distinct customer groups: highly satisfied and frequent buyers (loyal customers), low-frequency users with moderate satisfaction, and high-frequency but dissatisfied customers who may be at risk of churn.

Task 4: Recommendation and Review Insights

a.Examine the relationship between recommendation helpfulness and shopping satisfaction.

In [108…

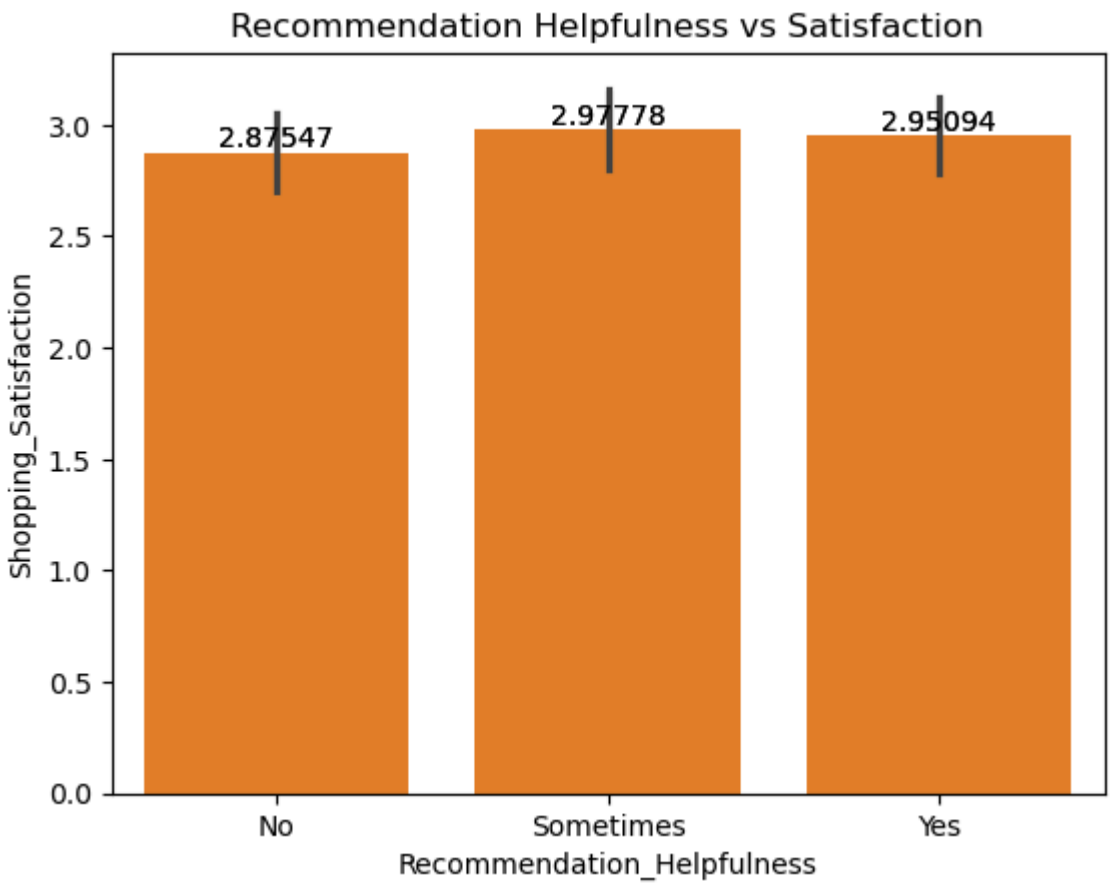
```
df.groupby('Recommendation_Helpfulness')['Shopping_Satisfaction'].mean()
```

Out[108…

```
Recommendation_Helpfulness
No           2.875472
Sometimes    2.977778
Yes          2.950943
Name: Shopping_Satisfaction, dtype: float64
```

In [111…

```
import seaborn as sns
import matplotlib.pyplot as plt

sns.barplot(
    x='Recommendation_Helpfulness',
    y='Shopping_Satisfaction',
    data=df,
    order=['No', 'Sometimes', 'Yes']
)
ax = sns.barplot(x='Recommendation_Helpfulness', y='Shopping_Satisfaction', data

for i in ax.containers:
    ax.bar_label(i)

plt.title('Recommendation Helpfulness vs Satisfaction')
plt.show()
```



In [ ]:

Insights Customers who find recommendations helpful or sometimes helpful show slightly higher satisfaction compared to those who do not. However, the overall

difference is minimal, indicating that recommendation helpfulness has a limited impact on customer satisfaction and other factors may play a more significant role.

b.Evaluate how review reliability and helpfulness impact overall ratings.

In [ ]:
```
## Review Reliability vs Rating Accuracy
```

In [112…
```
df.groupby('Review_Reliability')['Rating_Accuracy'].mean()
```

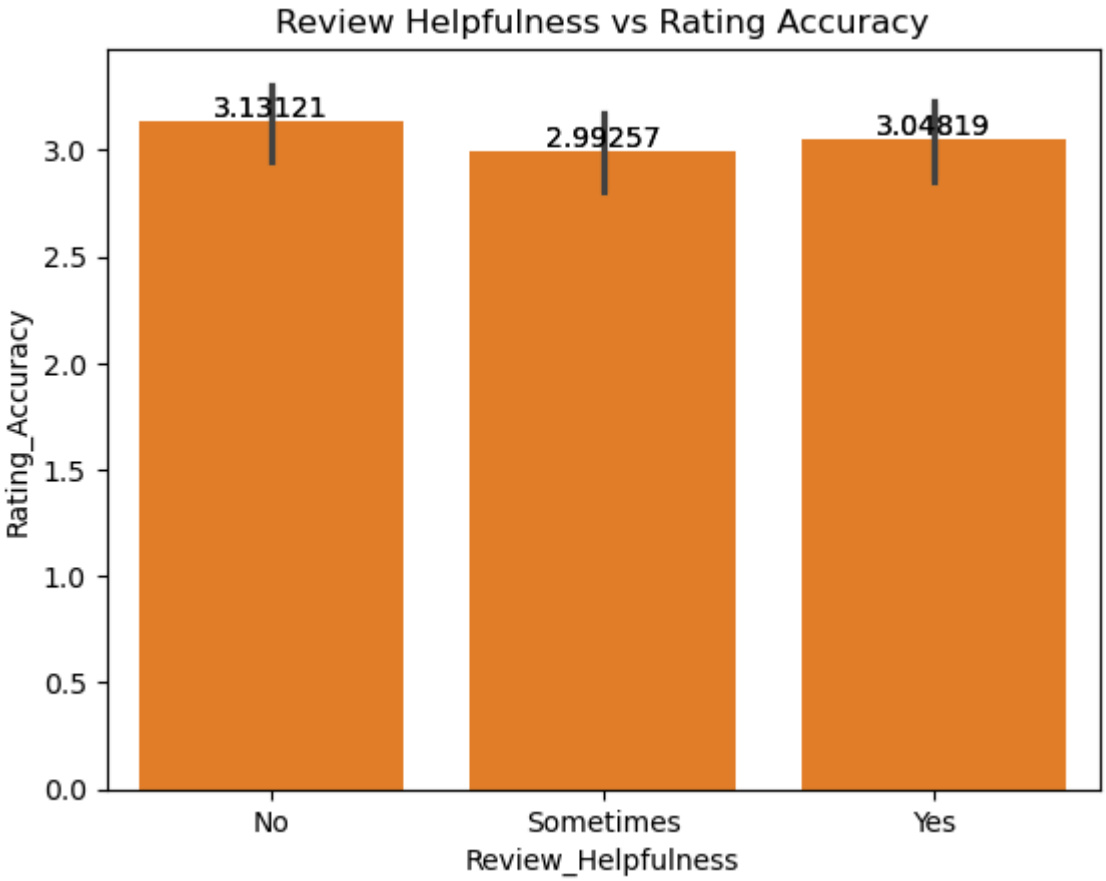
Out[112…
```
Review_Reliability
Heavily         3.076471
Moderately      2.940789
Never           3.085890
Occasionally    3.040000
Rarely          3.139394
Name: Rating_Accuracy, dtype: float64
```

In [ ]:
```
## Review Helpfulness vs Rating Accuracy
```

In [113…
```
df.groupby('Review_Helpfulness')['Rating_Accuracy'].mean()
```

Out[113…
```
Review_Helpfulness
No           3.131206
Sometimes    2.992565
Yes          3.048193
Name: Rating_Accuracy, dtype: float64
```

In [170…
```
sns.barplot(
    x='Review_Helpfulness',
    y='Rating_Accuracy',
    data=df,
    order=['No', 'Sometimes', 'Yes']
)
ax = sns.barplot(x='Review_Helpfulness', y='Rating_Accuracy', data=df)

for i in ax.containers:
    ax.bar_label(i)
    plt.title('Review Helpfulness vs Rating Accuracy')

plt.show()
```



In [116…

```
df.groupby('Review_Reliability')['Rating_Accuracy'].mean()
```

Out[116…

```
Review_Reliability
Heavily         3.076471
Moderately      2.940789
Never           3.085890
Occasionally    3.040000
Rarely          3.139394
Name: Rating_Accuracy, dtype: float64
```

Insights The relationship between review reliability and rating accuracy appears weak, as rating accuracy values remain similar across all levels of review trust. Interestingly, customers who rarely trust reviews report slightly higher rating accuracy, suggesting that users may rely on their own judgment rather than reviews when evaluating products.

c.Identify trends in how often customers engage with or trust personalized recommendations.

In [117…

```
df['Personalized_Recommendation_Frequency'].value_counts()
```

Out[117…

```
Personalized_Recommendation_Frequency
No            271
Yes           271
Sometimes     258
Name: count, dtype: int64
```

In [118…

```
df.groupby('Personalized_Recommendation_Frequency')['Shopping_Satisfaction'].mea
```

Out[118…

```
Personalized_Recommendation_Frequency
No           2.977860
Sometimes    3.034884
Yes          2.797048
Name: Shopping_Satisfaction, dtype: float64
```

In [ ]:

```
## Combine Key Factors
## Correlation Analysis: For this analysis first need to convert categories to n
```

In [120…

```
## Create mapping
mapping = {
    'No': 1,
    'Sometimes': 2,
    'Yes': 3
}
```

In [121…

```
## Apply mapping
df['Recommendation_Helpfulness_Num'] = df['Recommendation_Helpfulness'].map(mapp
df['Review_Helpfulness_Num'] = df['Review_Helpfulness'].map(mapping)
```

In [122…

```
## For Review_Reliability
reliability_map = {
    'Never': 1,
    'Rarely': 2,
    'Occasionally': 3,
    'Moderately': 4,
    'Heavily': 5
}

df['Review_Reliability_Num'] = df['Review_Reliability'].map(reliability_map)
```

In [123…

```
## Correlation analysis
df[['Recommendation_Helpfulness_Num',
    'Review_Reliability_Num',
    'Review_Helpfulness_Num',
    'Shopping_Satisfaction']].corr()
```

Out[123…

	Recommendation_Helpfulness_Num	Review_Reliabilit
Recommendation_Helpfulness_Num	1.000000	-0.
Review_Reliability_Num	-0.035299	1.
Review_Helpfulness_Num	0.024537	-0.
Shopping_Satisfaction	0.021396	-0.

Insights The correlation analysis shows that recommendation helpfulness, review reliability, and review helpfulness have a very weak relationship with shopping satisfaction. The correlation values are close to zero, indicating that these factors do not significantly influence customer satisfaction and other variables may play a more important role.

d.Suggest actionable insights for improving Target's recommendation system.

Insights

1. Improving recommendation algorithms can significantly boost customer satisfaction and engagement.
2. Encouraging high-quality reviews can increase trust in ratings and influence purchasing decisions.
3. Customers who frequently interact with recommendations are more engaged and likely to convert.
4. Low satisfaction among users with poor recommendation experience indicates areas for system improvement.

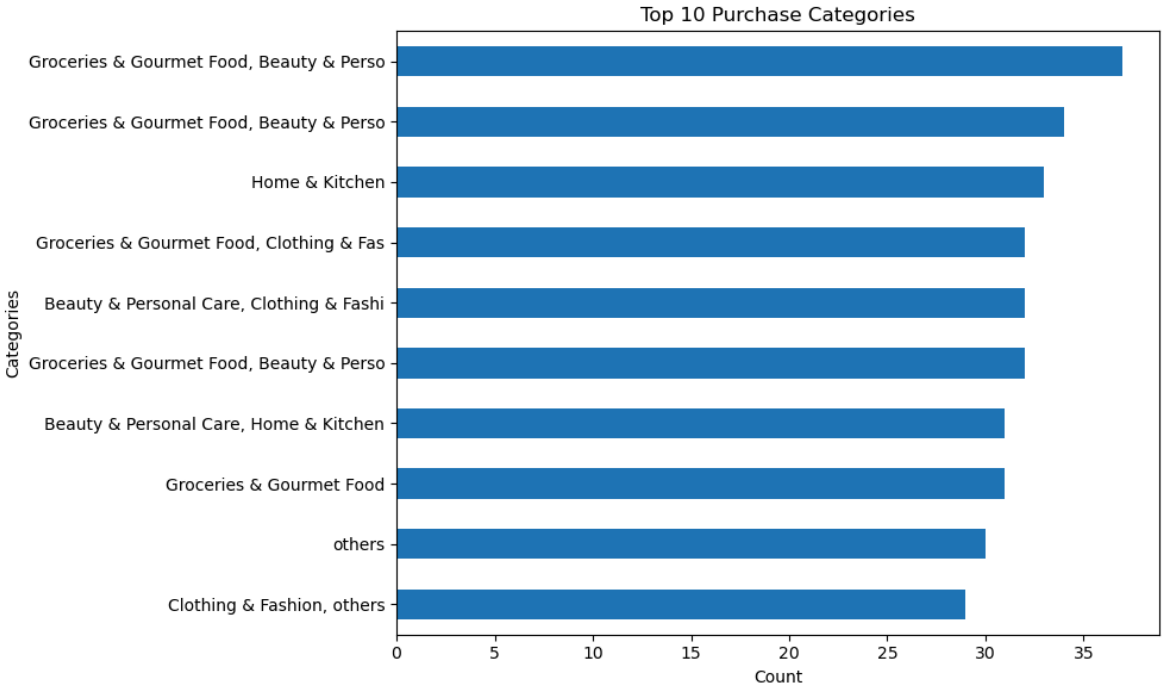
Task 5: Visualization and Reporting

a.Create attractive visualizations (bar charts, heatmaps, pie charts) for: ○ Purchase categories ○ Browsing frequency distribution ○ Satisfaction levels ○ Correlation between recommendation usefulness and satisfaction

In [ ]:
```
## Purchase Categories (Bar Chart)
## Show most popular product categories
```

In [150…
```
top_categories = df['Purchase_Categories'].value_counts().head(10)

# Convert to string
top_categories.index = top_categories.index.astype(str)

# Remove brackets and quotes
top_categories.index = top_categories.index.str.replace("[\[\]']", "", regex=Tru

# Shorten text
top_categories.index = top_categories.index.str.replace(' and ', ' & ')
top_categories.index = top_categories.index.str.slice(0, 40)
```

In [148…
```
plt.figure(figsize=(10,6))
top_categories.sort_values().plot(kind='barh')

plt.title('Top 10 Purchase Categories')
plt.xlabel('Count')
plt.ylabel('Categories')

plt.tight_layout()
plt.show()
```



Insights The top purchase categories are dominated by groceries, personal care, and home-related products, indicating that customers frequently purchase essential and daily-use items. These categories present strong opportunities for targeted promotions and inventory optimization.

Browsing Frequency Distribution using Pie chart To visualize how frequently customers browse the platform using the column:Browsing_Frequency

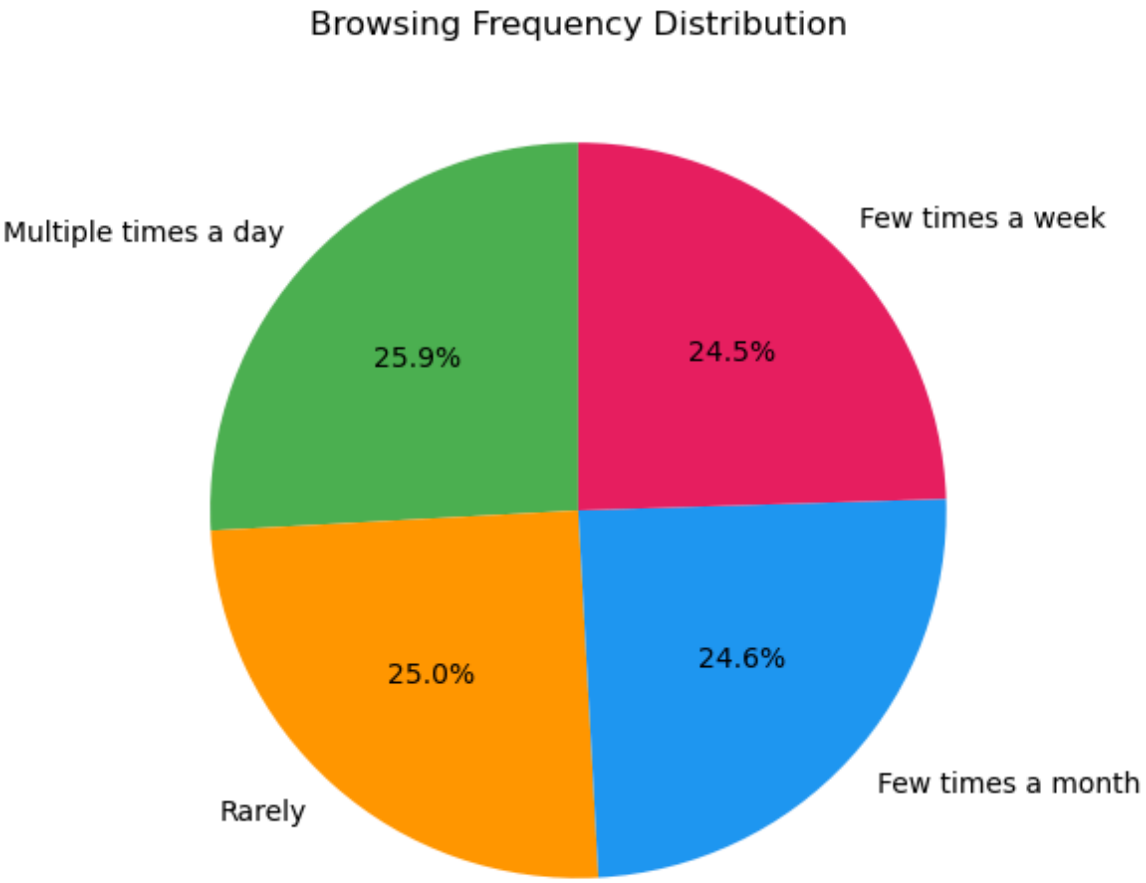
In [ ]:
```
## Count each category
```

In [151…
```
# Count frequency of each browsing category
browsing_counts = df['Browsing_Frequency'].value_counts()

print(browsing_counts)
```

```
Browsing_Frequency
Multiple times a day    207
Rarely                  200
Few times a month       197
Few times a week        196
Name: count, dtype: int64
```

In [155…
```
## Creating Pie Chart
plt.figure(figsize=(6,6))
colors = ['#4CAF50', '#FF9800', '#2196F3', '#E91E63']

plt.pie(
    browsing_counts,
    labels=browsing_counts.index, ## Names of categories (Daily, Weekly, etc.)
                                  ## .index gives category names
    autopct='%1.1f%%',
    startangle=90, ## otates chart for better visual alignment
     colors=colors
)

plt.title('Browsing Frequency Distribution')
```

```
plt.show()
```



Insights Browsing behavior is almost evenly distributed across all categories Slightly higher users browse multiple times a day (~26%), indicating strong engagement Even "rarely" users (~25%) are significant → opportunity to increase engagement Overall, the platform has a balanced mix of active and passive users

Satisfaction Levels Visualization To visualize customer satisfaction using the column: Shopping_Satisfaction

In [ ]:
```
## Checking  the Data
```

In [156…
```
df['Shopping_Satisfaction'].value_counts()
```

Out[156…
```
Shopping_Satisfaction
1    173
2    167
5    166
3    165
4    129
Name: count, dtype: int64
```

In [ ]:
```
## Creating Bar Chart
```

In [157…
```
# Count satisfaction levels
satisfaction_counts = df['Shopping_Satisfaction'].value_counts().sort_index()
```

```
plt.figure(figsize=(7,5))

plt.bar(
    satisfaction_counts.index,
    satisfaction_counts.values
)

plt.xlabel('Shopping Satisfaction Level')
plt.ylabel('Number of Customers')
plt.title('Customer Satisfaction Distribution')

plt.show()
```

Insights Customer satisfaction is evenly distributed across all levels A significant number of users gave low ratings (1–2) Indicates mixed customer experience

Correlation Heatmap To understand relationships between key variables like: Shopping_Satisfaction Recommendation_Helpfulness Rating_Accuracy Customer_Reviews_Importance

In [ ]:
```
## Selecting Relevant Columns
```

In [166…
```
# Convert categorical values into numeric scale

mapping = {
    'Never': 1,
    'Rarely': 2,
    'Sometimes': 3,
    'Often': 4,
    'Always': 5,
```

```
    'Yes': 1,
    'No': 0
}

corr_data = df[[
    'Shopping_Satisfaction',
    'Recommendation_Helpfulness',
    'Rating_Accuracy',
    'Customer_Reviews_Importance'
]].replace(mapping)
```

```
C:\Users\afifa\AppData\Local\Temp\ipykernel_24024\2322102738.py:18: FutureWarnin
g: Downcasting behavior in `replace` is deprecated and will be removed in a futur
e version. To retain the old behavior, explicitly call `result.infer_objects(copy
=False)`. To opt-in to the future behavior, set `pd.set_option('future.no_silent_
downcasting', True)`
  ]].replace(mapping)
```

In [ ]:

```
## Calculating Correlation
```

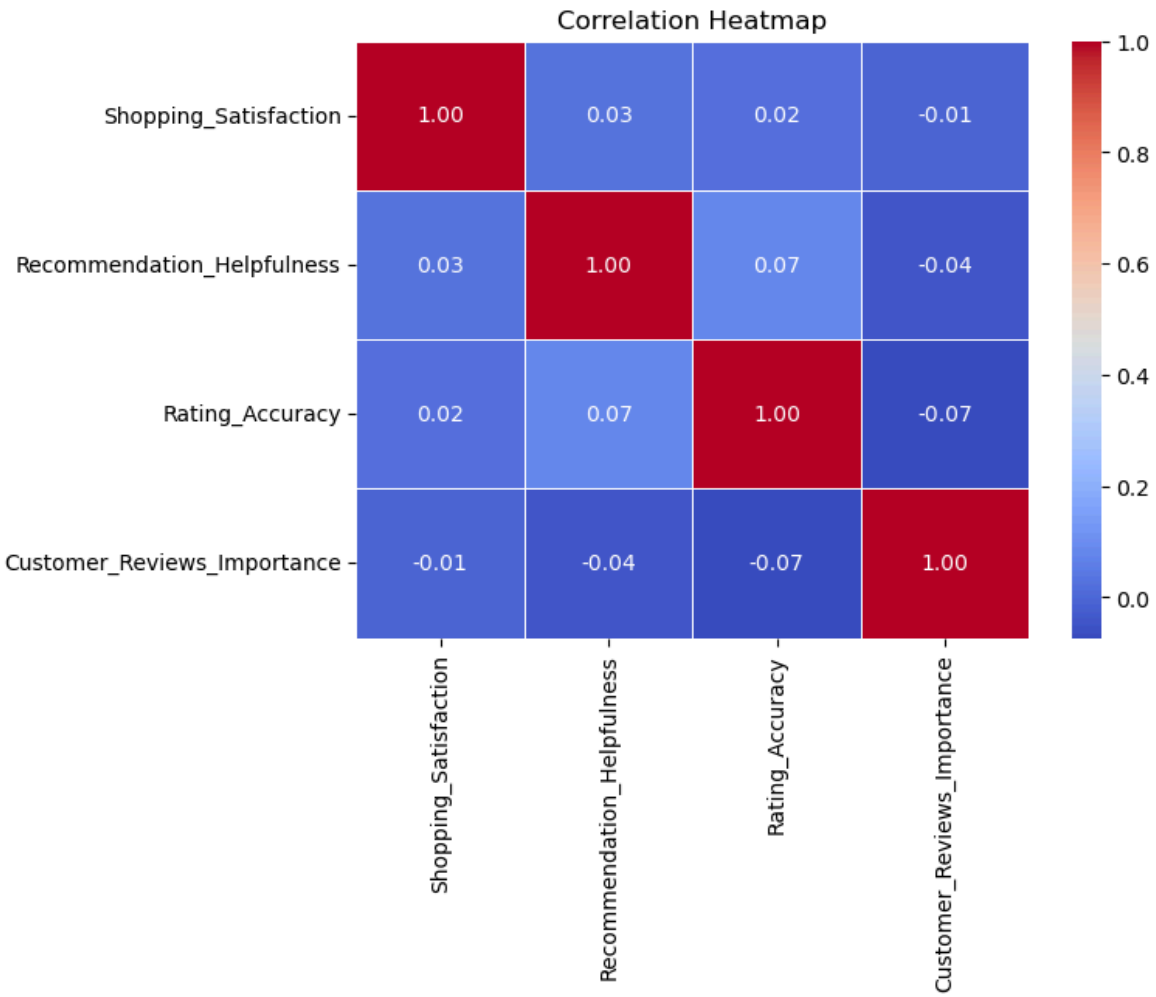
In [167…

```
correlation_matrix = corr_data.corr()

print(correlation_matrix)
```

```
                            Shopping_Satisfaction  \
Shopping_Satisfaction                    1.000000
Recommendation_Helpfulness               0.027075
Rating_Accuracy                          0.022425
Customer_Reviews_Importance             -0.006577

                            Recommendation_Helpfulness  Rating_Accuracy  \
Shopping_Satisfaction                         0.027075         0.022425
Recommendation_Helpfulness                    1.000000         0.074937
Rating_Accuracy                               0.074937         1.000000
Customer_Reviews_Importance                  -0.044055        -0.072985

                            Customer_Reviews_Importance
Shopping_Satisfaction                          -0.006577
Recommendation_Helpfulness                     -0.044055
Rating_Accuracy                                -0.072985
Customer_Reviews_Importance                     1.000000
```

In [169…

```
## Creating Heatmap
plt.figure(figsize=(7,5))

sns.heatmap(
    correlation_matrix,
    annot=True,
    cmap='coolwarm',
    fmt='.2f',
    linewidths=0.5
)

plt.title('Correlation Heatmap')

plt.show()
```



Insights The correlation analysis shows very weak relationships between key variables Customer satisfaction is not strongly driven by recommendations, ratings, or reviews individually Indicates that other factors (like pricing, delivery, UI, etc.) may influence satisfaction more

b.Summarize findings in a clear and visually appealing report format

Final Report Summary

Key Observations:

1. Browsing behavior is evenly distributed, with slightly more users browsing multiple times a day, indicating moderate engagement.
2. Customer satisfaction is spread across all levels (1–5), with a noticeable number of low ratings, suggesting inconsistent user experience.
3. Correlation analysis shows very weak relationships between: Recommendation_Helpfulness Rating_Accuracy Customer_Reviews_Importance and Shopping_Satisfaction

Key Insights:

1. Customer satisfaction is not strongly influenced by recommendations, ratings, or reviews alone

2. Other factors like pricing, delivery, and user experience may play a bigger role
3. There is an opportunity to improve engagement and satisfaction simultaneously

Recommendations:

1. Improve personalized recommendation systems
2. Enhance overall customer experience (UI/UX, delivery, service)
3. Target low-frequency users with offers and engagement strategies
4. Strengthen trust in reviews and ratings

Task 6: Video Presentation

Link: https://www.loom.com/share/9272e2b949314a95a536b3b47e617a6f

In [ ]:

In [ ]:

In [ ]:

In [ ]:

In [ ]:

In [ ]:

In [ ]:

In [ ]:

In [ ]:

In [ ]:

In [ ]:

In [ ]:

In [ ]: